

```

//
=====
=====
// EE180 Lab 3
//
// Decode module. Determines what to do with an instruction.
//
=====
=====

`include "mips_defines.v"

module decode (
    input [31:0] pc,
    input [31:0] instr,
    input [31:0] rs_data_in,
    input [31:0] rt_data_in,

    output wire [4:0] reg_write_addr,
    output wire jump_branch,
    output wire jump_target,
    output wire jump_reg,
    output wire [31:0] branch_addr,
    output wire [31:0] jr_pc,
    output reg [3:0] alu_opcode,
    output wire [31:0] alu_op_x,
    output wire [31:0] alu_op_y,
    output wire mem_we,
    output wire [31:0] mem_write_data,
    output wire mem_read,
    output wire mem_byte,
    output wire mem_half,
    output wire mem_signextend,
    output wire reg_we,
    output wire movn,
    output wire movz,
    output wire [4:0] rs_addr,
    output wire [4:0] rt_addr,
    output wire atomic_id,
    input atomic_ex,
    output wire mem_sc_mask_id,
    output wire mem_sc_id,

    output wire stall,

    input reg_we_ex,
    input [4:0] reg_write_addr_ex,
    input [31:0] alu_result_ex,
    input mem_read_ex,

```

```

    input reg_we_mem,
    input [4:0] reg_write_addr_mem,
    input [31:0] reg_write_data_mem

);

//
*****
*****
// instruction field
//
*****
*****

    wire [5:0] op = instr[31:26];
    assign rs_addr = instr[25:21];
    assign rt_addr = instr[20:16];
    wire [4:0] rd_addr = instr[15:11];
    wire [4:0] shamt = instr[10:6];
    wire [5:0] funct = instr[5:0];
    wire [15:0] immediate = instr[15:0];

    wire [31:0] rs_data, rt_data;
    wire signed [31:0] rs_data_signed = rs_data;

//
*****
*****
// branch instructions decode
//
*****
*****

    wire isBEQ      = (op == `BEQ);
    wire isBGEZNL   = (op == `BLTZ_GEZ) & (rt_addr == `BGEZ);
    wire isBGEZAL   = (op == `BLTZ_GEZ) & (rt_addr == `BGEZAL);
    wire isBGTZ     = (op == `BGTZ) & (rt_addr == 5'b00000);
    wire isBLEZ     = (op == `BLEZ) & (rt_addr == 5'b00000);
    wire isBLTZNL   = (op == `BLTZ_GEZ) & (rt_addr == `BLTZ);
    wire isBLTZAL   = (op == `BLTZ_GEZ) & (rt_addr == `BLTZAL);
    wire isBNE      = (op == `BNE);
    wire isBranchLink = (isBGEZAL | isBLTZAL);

//
*****
*****
// jump instructions decode
//
*****
*****

```

```
wire isJ      = (op == `J);
wire isJr     = (op == `SPECIAL) && (funct == `JR);
wire isJal    = (op == `JAL);
wire isJalr   = (op == `SPECIAL) && (funct == `JALR);
```

```
wire [4:0] link_reg = isJal ? 5'd31 : rd_addr;
wire [31:0] link_addr = pc + 32'd8;
```

//

// shift instruction decode

//

```
wire isSLL = (op == `SPECIAL) & (funct == `SLL);
wire isSRL = (op == `SPECIAL) & (funct == `SRL);
wire isSLLV = (op == `SPECIAL) & (funct == `SLLV);
wire isSRLV = (op == `SPECIAL) & (funct == `SRLV);
wire isSRA = (op == `SPECIAL) & (funct == `SRA);
wire isSRAV = (op == `SPECIAL) & (funct == `SRAV);
```

```
wire isShiftImm = isSLL | isSRL | isSRA;
wire isShift = isShiftImm | isSLLV | isSRLV | isSRAV;
```

//

// ALU instructions decode / control signal for ALU datapath

//

```
always @* begin
    casex({op, funct})
        {\`SPECIAL, \`SRAV}:    alu_opcode = \`ALU_SRA;
        {\`SPECIAL, \`SRA}:    alu_opcode = \`ALU_SRA;
        {\`XORI, \`DC6}:       alu_opcode = \`ALU_XOR;
        {\`ADDI, \`DC6}:       alu_opcode = \`ALU_ADD;
        {\`ADDIU, \`DC6}:      alu_opcode = \`ALU_ADDU;
        {\`SLTI, \`DC6}:       alu_opcode = \`ALU_SLT;
        {\`SLTIU, \`DC6}:      alu_opcode = \`ALU_SLTU;
        {\`ANDI, \`DC6}:       alu_opcode = \`ALU_AND;
        {\`ORI, \`DC6}:        alu_opcode = \`ALU_OR;
        {\`LB, \`DC6}:         alu_opcode = \`ALU_ADD;
        {\`LW, \`DC6}:         alu_opcode = \`ALU_ADD;
```

```

        {\`LBU, \`DC6}:      alu_opcode = \`ALU_ADD;
        {\`LH,  \`DC6}:      alu_opcode = \`ALU_ADD;
        {\`SB,  \`DC6}:      alu_opcode = \`ALU_ADD;
        {\`SH,  \`DC6}:      alu_opcode = \`ALU_ADD;
        {\`SW,  \`DC6}:      alu_opcode = \`ALU_ADD;
        {\`BEQ, \`DC6}:      alu_opcode = \`ALU_SUBU;
        {\`BNE, \`DC6}:      alu_opcode = \`ALU_SUBU;
        {\`SPECIAL, \`ADD}:   alu_opcode = \`ALU_ADD;
        {\`SPECIAL, \`ADDU}:   alu_opcode = \`ALU_ADDU;
        {\`SPECIAL, \`SUB}:    alu_opcode = \`ALU_SUB;
        {\`SPECIAL, \`SUBU}:   alu_opcode = \`ALU_SUBU;
        {\`SPECIAL, \`AND}:    alu_opcode = \`ALU_AND;
        {\`SPECIAL, \`OR}:     alu_opcode = \`ALU_OR;
        {\`SPECIAL, \`MOVN}:   alu_opcode = \`ALU_PASSX;
        {\`SPECIAL, \`MOVZ}:   alu_opcode = \`ALU_PASSX;
        {\`SPECIAL, \`SLT}:    alu_opcode = \`ALU_SLT;
        {\`SPECIAL, \`SLTU}:   alu_opcode = \`ALU_SLTU;
        {\`SPECIAL, \`SLL}:    alu_opcode = \`ALU_SLL;
        {\`SPECIAL, \`SRL}:    alu_opcode = \`ALU_SRL;
        {\`SPECIAL, \`SLLV}:   alu_opcode = \`ALU_SLL;
        {\`SPECIAL, \`SRLV}:   alu_opcode = \`ALU_SRL;
        {\`SPECIAL, \`NOR}:    alu_opcode = \`ALU_NOR;
        {\`SPECIAL, \`XOR}:    alu_opcode = \`ALU_XOR;
        {\`SPECIAL2, \`MUL}:   alu_opcode = \`ALU_MUL;
        // compare rs data to 0, only care about 1 operand
        {\`BGTZ, \`DC6}:      alu_opcode = \`ALU_PASSX;
        {\`BLEZ, \`DC6}:      alu_opcode = \`ALU_PASSX;
        {\`BLTZ_GEZ, \`DC6}:  begin
            if (isBranchLink)
                alu_opcode = \`ALU_PASSY; // pass link address for
mem stage
            else
                alu_opcode = \`ALU_PASSX;
        end
        // pass link address to be stored in $ra
        {\`JAL,  \`DC6}:      alu_opcode = \`ALU_PASSY;
        {\`SPECIAL, \`JALR}:   alu_opcode = \`ALU_PASSY;
        // or immediate with 0
        {\`LUI,  \`DC6}:      alu_opcode = \`ALU_PASSY;
        default:             alu_opcode = \`ALU_PASSX;
    endcase
end

//
*****
*****
// Compute value for 32 bit immediate data
//
*****
*****

```

```

    wire use_imm = &{op != `SPECIAL, op != `SPECIAL2, op != `BNE, op != `BEQ}; // where to get 2nd ALU operand from: 0 for RtData, 1 for Immediate

    wire [31:0] imm_sign_extend = {{16{immediate[15]}}}, immediate};
    wire [31:0] imm_zero_extend = {{16{1'b0}}}, immediate};
    wire [31:0] imm_upper = {immediate, 16'b0};

    wire [31:0] imm = (op == `LUI) ? imm_upper :
        (op == `ORI | op == `XORI) ? imm_zero_extend :
        imm_sign_extend;

//
*****
*****
// forwarding and stalling logic
//
*****
*****

    wire forward_rs_mem = &{rs_addr == reg_write_addr_mem, rs_addr != `ZERO, reg_we_mem};
    wire forward_rt_mem = &{rt_addr == reg_write_addr_mem, rt_addr != `ZERO, reg_we_mem};
    wire forward_rs_ex = &{rs_addr == reg_write_addr_ex, rs_addr != `ZERO, reg_we_ex};
    wire forward_rt_ex = &{rt_addr == reg_write_addr_ex, rt_addr != `ZERO, reg_we_ex};

    assign rs_data = forward_rs_ex ? alu_result_ex : (forward_rs_mem ? reg_write_data_mem : rs_data_in);
    assign rt_data = forward_rt_ex ? alu_result_ex : (forward_rt_mem ? reg_write_data_mem : rt_data_in);

    wire rs_mem_dependency = &{rs_addr == reg_write_addr_ex, mem_read_ex, rs_addr != `ZERO};
    wire rt_mem_dependency = &{rt_addr == reg_write_addr_ex, mem_read_ex, rt_addr != `ZERO};

    wire isLUI = op == `LUI;
    wire read_from_rs = ~|{isLUI, jump_target, isShiftImm};

    wire isALUImm = |{op == `ADDI, op == `ADDIU, op == `SLTI, op == `SLTIU, op == `ANDI, op == `ORI};
    wire read_from_rt = ~|{isLUI, jump_target, isALUImm, mem_read};

    assign stall = (rs_mem_dependency & read_from_rs) |
        (rt_mem_dependency & read_from_rt);

```

```

    assign jr_pc = rs_data;
    assign mem_write_data = /*(isJal | isJalr) ? link_addr :*/
rt_data;

//
*****

    // for shift operations, use either shamt field or lower 5 bits of
rs
    // otherwise use rs

    wire [31:0] shift_amount = isShiftImm ? shamt : rs_data[4:0];
    assign alu_op_x = isShift ? shift_amount : rs_data;

    // for link operations, use next pc (current pc + 8)
    // for immediate operations, use Imm
    // otherwise use rt

    assign alu_op_y = (isJal | isJalr) ? pc + 32'd8 :
        (use_imm) ? imm : rt_data;
    assign reg_write_addr = (isJal | isJalr) ? link_reg : ((use_imm) ?
rt_addr : rd_addr);

    // determine when to write back to a register (any operation that
isn't an
    // unconditional store, non-linking branch, or non-linking jump)
    assign reg_we = ~|{(mem_we & (op != `SC)), isBGEZNL, isBGTZ,
isBLEZ, isBLTZNL, isBNE, isBEQ, isJ};

    // determine whether a register write is conditional
    assign movn = &{op == `SPECIAL, funct == `MOVN};
    assign movz = &{op == `SPECIAL, funct == `MOVZ};

//
*****

// Memory control
//
*****

    assign mem_we = |{op == `SW, op == `SB, op == `SC, op ==
`SH}; // write to memory
    assign mem_read = |{op == `LW, op == `LL, op == `LB, op == `LBU,
op == `LH}; // use memory data for writing to a register
    assign mem_byte = |{op == `SB, op == `LB, op == `LBU}; //
memory operations use only one byte
    assign mem_signextend = ~|{op == `LBU}; // sign extend sub-
word memory reads

```

```

    assign mem_half = |{op == `SH, op == `LH};

//
*****
*****
// Load linked / Store conditional
//
*****
*****
    assign mem_sc_id = (op == `SC);

    // 'atomic_id' is high when a load-linked has not been followed by
    a store.
    assign atomic_id = (op == `LL) ? 1'b1 :
        (op == `SC) ? 1'b0 : atomic_ex;

    // 'mem_sc_mask_id' is high when a store conditional should not
    store
    // in other words, high when an ll has been followed by a sc,
    // or when ll has not been called.
    assign mem_sc_mask_id = mem_sc_id & (~atomic_id);
    // assign mem_sc_mask_id = 1'b0;

//
*****
*****
// Branch resolution
//
*****
*****

wire isEqual = rs_data == rt_data;
wire isZero = rs_data == 32'b0;
wire isGEZero = rs_data[31] == 0;
wire isLTZero = rs_data[31] == 1;

assign jump_branch = |{isBEQ & isEqual, // true if should branch
    isBNE & ~isEqual,
    isBGEZNL & isGEZero,
    isBLEZ & (isLTZero | isZero),
    isBLTZNL & isLTZero,
    isBGTZ & isGEZero & ~isZero};

assign jump_target = |{isJ, isJal};
assign jump_reg = |{isJr, isJalr};
assign branch_addr = (imm_sign_extend << 2) + pc + 3'h4;

```

```
    //wire [31:0] imm_sign_extend = {{16{immediate[15]}}}, immediate};  
endmodule
```